MCS-322	DESIGN AND ANALYSIS OF ALGORITHMS	L T P 3 1 0	4 Credits

Course Objectives:

CO1	To understand and remember algorithms and its analysis procedure.
CO2	To introduce the concept of data structures through ADT including List, Stack, Queues. Analyze the asymptotic performance of algorithms.
CO3	To design and implement various data structures and algorithms.
CO4	Apply important algorithmic design paradigms and methods of analysis.
CO5	Synthesize efficient algorithms in common engineering design situations.

MODULE I

Algorithm Analysis and Review of Data Structures: Algorithms, Psuedo code for expressing algorithms, Performance Analysis-time complexity and space complexity-notation, Omega notation and Theta notation, little o notation, Probabilistic analysis, Amortized analysis, Review of Data Structures- The List ADT, Stack ADT, Queue ADT, Implementations using template class, Hash Functions, Collision Resolution in hashing, Priority queues-Definition, Priority queues-ADT, Heaps-Definition, Insertion and Deletion, Applications-Heap sort.

MODULE II

Divide and conquer: General method, applications-Binary search, Quick sort, Merge sort, Strassen's Matrix Multiplication.

Greedy method: General method, applications-Job sequencing with dead lines, 0/1 knapsack problem, Minimum cost spanning trees, Single source shortest path problem. Dynamic Programming: General method, applications-Matrix chain multiplication, Optimal binary search trees, 0/1 knapsack problem, All pairs shortest path problem, Traveling sales person problem, Reliability design.

MODULE III

Searching and Traversal Techniques: Efficient non-recursive Tree Traversal Algorithms, DFS, BFS of Graphs, AND/OR graphs, game trees, Search Trees-Balanced search trees-AVL trees, representation, Operations-insertion, deletion and searching, B-Trees-B-Tree of order m, Operations- insertion, deletion and searching.

Backtracking and Branch and Bound: General method (Backtracking), Applications-n-queen problem, sum of subsets problem, graph coloring, Hamiltonian cycles. General method (Branch and Bound), Applications - Traveling sales person problem, 0/1 knapsack problem-LC Branch and Bound solution.

NP-Hard and NP-Complete problems: Basic concepts, non-deterministic algorithms, NP - Hard and NP- Complete classes, Cook's theorem.

References:

1. *E. Horowitz, S.Sahani and S.Rajasekharan, "Computer Algorithms/C++", Galgotia Publishers pvt. Limited.*

2. *Mark Allen Weiss, "Data Structures and Algorithm Analysis in C++", 2nd Edition, Pearson Education.*
3. *T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, "Introduction to Algorithms", 2nd Edition, PHI Pvt.Ltd./ Pearson Education.*
4. *Aho, Ullman and Hopcroft, "Design and Analysis of algorithms", Pearson Education.*
5. *A. Levitin, "Introduction to the Design and Analysis of Algorithms", Pearson Education.*
6. *S. Sahni, "Data structures, Algorithms and Applications in C", University press (India) pvt ltd, 2nd edition, Orient Longman pvt. ltd.*
7. *K. A. Berman, J. L.Paul, "Fundamentals of Sequential and Parallel Algorithms", Thomson.*
8. *Adam Drozdek, "Data Structures And Algorithms in C", 3rd Edition, Thomson.*
9. *M. T. Goodrich and R. Tomassia, "Algorithm Design: Foundations", Analysis and Internet examples, John Wiley and sons.*

Course Outcome:

CO1	For a given algorithms analyze worst-case running times of algorithms based on asymptotic analysis and justify the correctness of algorithms .
CO2	Describe the greedy paradigm and explain when an algorithmic design situation calls for it. For a given problem develop the greedy algorithms.
CO3	Describe the divide-and-conquer paradigm and explain when an algorithmic design situation calls for it. Synthesize divide-and-conquer algorithms. Derive and solve recurrence relation.
CO4	Describe the dynamic-programming paradigm and explain when an algorithmic design situation calls for it
CO5	For a given model engineering problem, model it using graph and write the corresponding algorithm to solve the problems.